

DRAW IT or LOSE IT!

Draw It or Lose It

CS 230 Project Software Design

Version 2.0

Company: Creative Technology Solutions (CTS)

Client: The Gaming Room

Table of Contents

Document Revision History 2
Executive Summary 3
Requirements 3
Design Constraints 3
System Architecture View 3
Domain Model 4
Evaluation 4
Recommendations 8

Document Revision History

Version	Date	Author	Comments
2.0	09/25/26	Roland Pratts	Updated software design document for Project Two, including platform evaluation and recommendations for the Draw It or Lose It application.

Executive Summary

The Gaming Room wants to expand its existing Android game, Draw It or Lose It, into a web-based application that can support users on multiple platforms. The application must allow one or more teams to participate in a game, allow multiple players on each team, and ensure that game and team names are unique. The system must also control how game instances are created so that the application can manage shared game data efficiently. To meet these requirements, the application will use object-oriented programming and common software design patterns. The Singleton pattern will be used in the GameService class to manage a single shared service instance, while the Iterator pattern will be used to search existing games, teams, and players before creating new objects. The design will also use inheritance through a shared Entity base class so that common attributes such as an ID and name do not have to be repeated in each class. This approach creates a more organized, maintainable, and scalable foundation for developing the web-based version of Draw It or Lose It.

Requirements

The Gaming Room requires Draw It or Lose It to support one or more teams in each game, with multiple players assigned to each team. Game names and team names must be unique so users can determine whether a name is already in use before creating a new game or team. The application must also use unique identifiers for each game, team, and player. The system will be developed as a web-based application so it can support users on multiple platforms instead of being limited to Android. The design must also ensure that only one GameService instance manages the game data in memory while still allowing multiple game objects to be created and managed within the application.

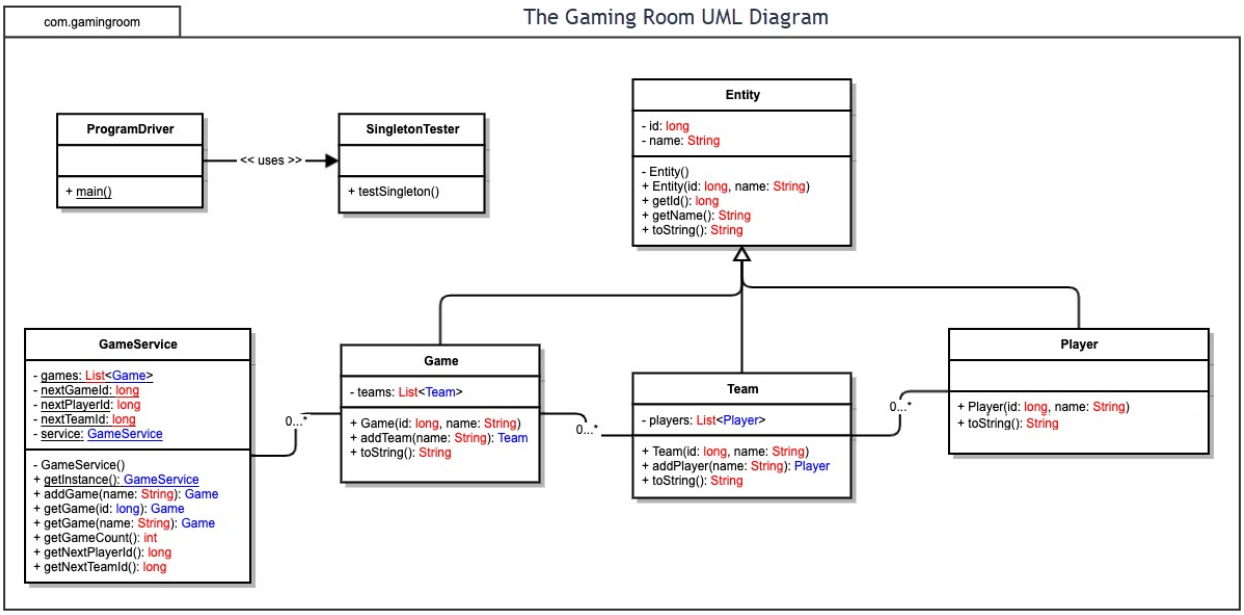
Design Constraints

The main design constraint is that Draw It or Lose It must operate in a web-based distributed environment and support users on multiple platforms. This means the application cannot depend on features that are available only on Android. The design should use platform-independent technologies so users on Windows, macOS, Linux, and mobile devices can access the game through a web browser. The application must also manage multiple games, teams, and players while keeping names and identifiers unique. Because the application will be distributed across a network, developers must consider reliable communication between the client and server, possible network delays or outages, and the security of user and game data. The design must also support future growth, so the code should be organized into reusable classes and use patterns such as Singleton and Iterator patterns to manage shared data and search collections efficiently. These constraints affect development because the application must be tested across different platforms and browsers while maintaining consistent behaviors and performance.

System Architecture View

Domain Model

The UML class diagram shows the main classes used in the Draw It or Lose It application and how they relate to one another. The Entity class is the base class and contains the common attributes id and name, along with methods such as getId(), getName(), and toString(). The Game, Team, and Player classes inherit from Entity, which demonstrates the object-oriented programming principle of inheritance. This prevents the same attributes and methods from being repeated in each class and makes the code easier to maintain. The GameService class manages a collection of Game objects and uses the Singleton pattern so only one GameService instance exists to manage the application data. A Game can contain multiple Team objects, and each Team can contain multiple Player objects, which matches the client requirement that a game can have one or more teams and that each team can have multiple players. The addGame(), addTeam(), and addPlayer() methods support the requirement that names remain unique by checking existing objects before creating new ones. The diagram also demonstrates encapsulation by keeping data inside classes and providing methods for controlled access to that data. Together, inheritance, encapsulation, the Singleton pattern, and the Iterator pattern create a design that is organized, reusable, and able to meet the client's software requirements efficiently.



Evaluation

Using your experience to evaluate the characteristics, advantages, and weaknesses of each operating platform (Linux, Mac, and Windows) as well as mobile devices, consider the requirements outlined below and articulate your findings for each. As you complete the table, keep in mind your client's requirements and look at the situation holistically, as it all has to work together.

Development Requirements	Mac	Linux	Windows	Mobile Devices
Server Side	macOS can host a web-based application using technologies such as Java, Apache, Nginx, and other web server software. It provides a stable Unix-based environment and includes strong security features. However, macOS is not commonly used for large-scale production web servers compared with Linux or Windows Server. Apple also requires macOS to run on Apple hardware, which may increase hardware costs. For The Gaming Room, macOS could support development and smaller deployments, but it would be less practical for hosting a game that may need to scale to thousands of users.	Linux is well suited for hosting a web-based application and is widely used for web servers. It supports technologies such as Java, Apache, Nginx, databases, containers, and cloud services. One major advantage is that most Linux distributions do not require an operating system licensing fee, which can reduce costs for The Gaming Room. Linux is also stable, secure, efficient, and easy to scale by adding more servers or cloud resources. A possible weakness is that administrators need knowledge of Linux commands, permissions, networking, and server configuration. Overall, Linux provides a strong server environment for a large distributed web application.	Windows Server can host the game application and provides tools for web hosting, networking, user management, and security. It can support Java applications as well as Microsoft technologies such as IIS and .NET. Windows Server also provides a graphical interface that may be easier for administrators who are familiar with Windows. A disadvantage is licensing cost because Windows Server is commercial software and may also require additional licenses depending on how it is deployed. It can support a large web application, but the licensing and resource requirements may make it more expensive than Linux.	Mobile operating systems such as Android and iOS are designed mainly to run client applications rather than host large production web servers. Although developers can run small local services on mobile devices, phones and tablets do not provide the processing power, storage, network reliability, or server management features needed to support thousands of game users. Because of these limitations, mobile devices should communicate with a separate server rather than host the server-side application. The main purpose of Android and iOS in this system would be to provide users with access to the game.

Client Side	Mac users can access Draw It or Lose It through modern web browsers such as Safari, Chrome, or Firefox. Developers should create the game with responsive HTML, CSS, and JavaScript so that the same web application works correctly across different screen sizes and browsers. Developers must test the application on macOS to make sure graphics, controls, networking, and browser features behave correctly. Because the application is web based, a separate Mac version may not be necessary, which can reduce development time and cost.	Linux users can access the application through browsers such as Chrome, Chromium, and Firefox. Developers need to use web standards and avoid features that only work on one operating system or browser. Testing is important because there are many Linux distributions and browser combinations. A responsive web design reduces the need to create a separate Linux application. This can save development time and allow the same client-side code to communicate with the REST API used by Windows, Mac, and mobile clients.	Windows supports several common web browsers, including Edge, Chrome, and Firefox. Developers should test Draw It or Lose It across these browsers and different screen resolutions to make sure the interface behaves consistently. Responsive HTML, CSS, and JavaScript can allow Windows users to use the same web application as Mac and Linux users. This approach reduces the need for separate desktop applications and allows developers to focus more time on one common client interface.	Mobile support requires developers to make sure the interface works properly on Android and iOS devices. The website should use responsive design, so buttons, images, text, and game controls adjust to different screen sizes and orientations. Developers also need to consider touch controls, slower mobile networks, battery usage, and different browser behavior. Testing should include both Android and iOS devices. More users could be added through a registration system that sends account information to the server and stores the information in a database. Future features could include player profiles, team invitations, score tracking, leaderboards, and game history.

Development Tools	Developers working on macOS can use Java, HTML, CSS, JavaScript, and development environments such as Eclipse, Visual Studio Code, or IntelliJ IDEA. Xcode would also be required if the company later develops a native iOS application. Xcode is available for macOS and provides tools for building and testing Apple applications. Many web development tools are available at no cost, although some professional development products may require paid licenses. Developers who support native Apple applications would also need knowledge of Swift and Apple's development tools.	Linux supports Java, HTML, CSS, JavaScript, Python, databases, and many other technologies used in web development. Developers can use Eclipse, Visual Studio Code, IntelliJ IDEA, Git, Docker, and command-line tools. Many Linux development tools are open source or available for free, which can reduce software costs. Developers may need stronger command-line and server administration skills compared with some other platforms. Linux is especially useful for the development team because the development environment can closely match the production server environment.	Windows developers can use Java, HTML, CSS, JavaScript, C#, and many other programming languages. Common tools include Eclipse, Visual Studio, Visual Studio Code, Git, and database management software. Some Microsoft development tools are available in free editions, while professional or enterprise editions may have licensing costs. Developers may need additional expertise if the application uses Windows-specific server technologies such as IIS or .NET. However, because the game is web based, much of the same Java and web development code can be shared with the other platforms.	Supporting Android and iOS may require additional tools and expertise if The Gaming Room decides to create native mobile applications. Android development commonly uses Android Studio with Java or Kotlin, while iOS development commonly uses Xcode with Swift. This can increase development time because developers must test different devices, operating system versions, screen sizes, and touch interfaces. The company may need developers with separate Android and iOS experience. A responsive web application can reduce this need because one HTML-based interface can work across many mobile devices. However, platform-specific development may still be needed if the company later wants features that require direct access to mobile hardware or operating system services.

Recommendations

Analyze the characteristics of and techniques specific to various systems architectures and make a recommendation to The Gaming Room. Specifically, address the following:

1. **Operating Platform**: I recommend that The Gaming Room use Linux as the primary server operating platform for the web-based version of Draw It or Lose It. Linux is a strong choice for hosting web applications because it is stable, secure, scalable, and commonly used in server environments. Most Linux distributions can also be used without operating system licensing fees, which can reduce costs for the company. The application can run on a Linux server while users access it through web browsers on Windows, macOS, Linux, Android, and iOS devices. Using a browser-based client allows The Gaming Room to support many operating environments without creating a separate version of the game for every platform.

2. **Operating Systems Architectures**: Linux supports several processor architectures, including x86-64 and ARM-based systems, which gives The Gaming Room flexibility when choosing physical servers, virtual machines, or cloud-based systems. Linux also uses a multiuser and multitasking architecture that allows multiple processes and users to operate at the same time. This is important for Draw It or Lose It because the server may need to handle many players and games simultaneously. The application can be developed using Java, which also improves portability because Java applications run inside the Java Virtual Machine. This allows much of the same server-side application code to run on different hardware without being rewritten for every processor architecture.

3. **Storage Management**: A relational database management system such as PostgreSQL or MySQL would be appropriate for storing the application's persistent data. The database could contain user accounts, teams, game information, scores, and game history. The actual stock images used by the game could be stored separately in file storage or cloud object storage, while the database keeps information about each image and where it is located. Linux supports commonly used file systems such as ext4, which provides reliable storage for the operating system and application files. Regular backups should also be created so important information can be restored if hardware, software, or user errors cause data loss.

4. **Memory Management**: Linux uses virtual memory to manage how applications use physical RAM. Each process receives its own protected memory space, which helps prevent one application from directly interfering with another process. The operating system can also move less frequently used information between RAM and secondary storage when necessary. For Draw It or Lose It, memory will be needed for active games, users, images, network connections, and server processes. Because the application is written in Java, the Java Virtual Machine also manages application memory. Java uses automatic garbage collection to reclaim memory from objects that are no longer being used. Developers should still design the application carefully so that unnecessary objects, sessions, or image data are not kept in memory longer than needed.

5. **Distributed Systems and Networks**: Draw It or Lose It can operate as a distributed system in which multiple client devices communicate with a central server over a network. Users on Windows, macOS, Linux, Android, and iOS can send requests to the server using a web browser. The clients can communicate with the server through a REST-style API using standard HTTP or

HTTPS requests. The server would handle game logic, authentication, user information, and access to stored data, while the clients would display the game's interface.

The network is an important dependency because clients must be able to communicate with the server for the game to function correctly. Developers should plan for slow connections, temporary outages, failed requests, and users disconnecting during a game. The application should provide useful error messages and should avoid losing important game information when possible. Server redundancy, load balancing, monitoring, and database backups could also improve availability as the number of players increases. If one server becomes unavailable, additional servers could help keep the service operating.

6. **Security**: Security should be included throughout the design of Draw It or Lose It. All communication between clients and the server should use HTTPS so information is encrypted while traveling across the network. User passwords should not be stored as plain text. Instead, passwords should be protected using a secure password hashing method with appropriate salts.

   The application should also use authentication to verify the identity of each user and authorization to control what the user is allowed to access. This follows the same idea used in the Module Four application, where different roles could be assigned different permissions. For example, regular users should only have access to normal game functions, while administrator functions should require additional permissions.

   Linux also provides security features such as file permissions, user and group accounts, process isolation, firewalls, and regular security updates. The server should only expose the network ports and services that are required by the application. The Gaming Room should also maintain regular backups, install security updates, monitor unusual activity, and limit administrator access. Combining operating system security with secure application development and encrypted network communication would help protect user information across all supported platforms.